Sabanci University



Operating Systems

CS 307

Programming Assignment - 2: Multi-Core Scheduling with Synchronization

Release Date: 27 March 2026
Deadline: 7 April 2026 23.55

1 Problem Description

In the lectures, we have seen that the OS scheduler aims to optimize *average turnaround time* (AvgTAT) for computationally heavy processes and *fairness* conditions for more interactive processes. However, these metrics are meaningful for only single core machines. Modern computers consist of multi-core CPUs and scheduling processes in a multi-core machine brings extra challenges and concerns.

Load balancing is one of the most important issues in multi-core scheduling. If all the jobs are assigned to a single core, even though the OS schedules all the jobs optimally on this core, we might not get the best performance and utilization of the whole CPU. A mediocre scheduling strategy that somehow evenly distributes jobs to cores but does not guarantee the minimum AvgTAT on each core might perform much better and yield a better overall AvgTAT than a scheduler that distributes jobs to cores poorly, but guarantees AvgTAT inside a core.

While achieving the best load balancing is desirable, one should consider an important problem. Distributing jobs evenly to CPU cores and improving overall CPU utilization might require frequent migration of jobs from one core to another during its execution. While a job is running on a core, this core caches important data and meta-data about this job which increases the runtime performance. If the job is scheduled into another core after a context switch, this job suffers from a problem called *cold start*. Since the new core contains no data and metadata of the new process in its cache, its performance in the new core is pretty low and it stays low until the new core's cache is populated with this new job's entries. Hence, frequent job exchange might degrade performance even though it increases the CPU utilization and load balancing. For more comprehensive information on challenges of multi-core scheduling and possible solutions, please read Chapter 10 of OSTEP before proceeding any further.

1.1 A Simple Multi-Core Scheduling Policy

A widely used approach to accommodate multiple cores at once is to have each of them store their tasks in a separate collection data structure. When a task is created, it is assigned (by some heuristic of the OS) to one of the cores and inserted into its collection, and from that point on that job only concerns the core it is designated to and we get all the performance benefits

of cache affinity.

However, being very firm about the job-processor matching assigned at the very beginning can be costly. Since we cannot guess the running times of jobs beforehand, some of the cores might finish their tasks before others. In this case, these cores stay idle which causes inefficiency and low CPU utilization. In this case, we might want to evenly distribute jobs of busy cores among idle cores, hence the load balancing.

One possible solution is to not render cores completely oblivious about other cores' tasks and allow the migration of tasks from core to core once in a while. This engenders two new problems, one of which is synchronization. Whenever we try to migrate one job from one core to another, we have to be certain that another core isn't also trying to grab that job. If we don't account for possible data races in the data structures of the cores, we might have duplicate jobs, or a lost job, or even a deadlock. Therefore, the collection data structure cores keep must be thread-safe.

Lastly, we have to keep in mind the cost of migrating a task between cores. When a task runs in a core, it builds up its own cache state on this core. The cache is a small local lookup table that the processor checks first for any data operation. If the data it looks for isn't there, it fetches data from the main memory. The cost of accessing the memory is astronomically high compared to fetching from the cache. After running on the same core for some time, most of the frequently used data and metadata of this job can be found in this core's cache. This is called *cache affinity* and improves the runtime performance of a job. Migrating a job from one core to another completely resets its built-up cache state and it is called a cold start. Until the job gains cache affinity, this job performs poorly which degrades the overall performance.

In this assignment, you will build a multi-core scheduler simulator where cores are going to be simulated by threads, and they are going to tackle some dummy jobs we assign. You will also implement a collection data structure that will be maintained by each core thread to keep track of its jobs. This collection data structure is going to be called `TaskDequeue` (or `TDQ` in short) from now on. Each core will insert and remove jobs to its `TDQ` while scheduling them and they will be able to fetch some jobs from other cores' `TDQ` 's for load balancing. Since multiple threads will be operating on the same `TDQ` , we require your `TDQ` implementation to work correctly while being accessed concurrently.

In the following chapter, we will provide requirements and details for the multi-core scheduler and the `TDQ` concurrent collection data structure. We will provide the API for the both the scheduler and `TDQ` i.e., the method signatures and their specifications.

2 API Methods and Specifications

In this section, we will guide you through the essential components of the multi-core scheduler simulator. We will start by explaining the data structures required, including the job representation and `TDQ` , followed by the API methods of `TDQ` you will need to implement. Afterward, we will discuss the core simulator threads, outlining their operations and responsibilities. Finally, we will cover synchronization issues and strategies for handling concurrent access to queues, with a focus on balancing performance and correctness.

2.1 Data Structures for Simulation

- **Task Structure**: This struct represents individual jobs, the primary units of work within the simulation. Each job will be stored in a `TDQ` , which organizes and manages job distribution across cores. We have provided the `Task` struct to represent each job in the file `multi_sched.h`. You don't have to implement it by yourself. Here is the definition:

```
typedef struct Task {
    char* task_id;
    int task_duration;
    double cache_warmed_up;
    TaskDeque* owner;

    pthread_mutex_t in_degree_lock;
    int in_degree;
    int num_dependents;
    struct Task** dependents;
} Task;
```

  - `task_id`: A unique identifier for each job. You can assume that uniqueness is ensured, as jobs will be given to you with unique IDs.

- `task_duration`: Represents the total time required to complete the job. The time unit is measured in CPU cycles, simulating processing time.
- `cache_warmed_up`: A measure of cache affinity for this job. Higher values indicate better cache optimization, reducing the required execution time due to improved cache utilization. You might think of this parameter as a coefficient that will be multiplied by the expected portion of the job to be completed in this scheduling round to compute actual completed portion. Higher the parameter, higher the cache hit rate and bigger the piece of job it completes.
- `owner`: Stores the information of which processor the task belonged to. Used in `executeJob` to update cache affinity.
- `in_degree_lock`: To atomically change the in degree of a task.
- `in_degree`: Represents the number of prerequisite tasks that must successfully complete before this task can be scheduled. Because multiple cores might finish prerequisite tasks simultaneously, this needs to be updated atomically. A task is only ready to be pushed to a queue when this value reaches exactly zero.
- `num_dependents`: An integer specifying the number of tasks that directly depend on the completion of this task. It defines the length of the `dependents` array.
- `dependents`: An array of pointers to other `Task` structures that rely on this task. When a core finishes executing this task, it must iterate through this array and atomically decrement the `in_degree` of every dependent task.

You are not allowed to modify this struct, as it is referenced throughout the simulation code. Jobs will be provided as input to your simulator, so you will not create jobs yourself. The method for obtaining jobs for the scheduler will be explained in following sections.

- `TDQ` **Structure**: To manage jobs for each core, you will implement the `TDQ` struct. This structure, defined in `multi_sched.h`, keeps jobs of the core as a collection object and organizes them in a way that is easy to use by the scheduler. You are responsible for declaring the fields and

state of the struct. It might be an array based or node based collection depending on your preferences. You might declare other helper structs in `multi_sched.h` if you need them.

Moreover, your `TDQ` library must provide some API methods that will be discussed in the next part.

2.2 API for TaskDequeue

We want `TDQ` to be a double-ended queue, however the scheduling policy followed by a single core for its own `TDQ` is a design choice and is up to you. Some options to choose from are STCF, where you must keep the `TDQ` sorted at all times, or Round-Robin where you can simply insert an unfinished job back to the end of the dequeue. You must try these and other options for how to reinsert unfinished jobs to your dequeues and these will have performance implications on the `vruntime` accumulated by your simulator. A part of your grade will depend on the performance of your simulator.

There are three primary API functions manage task processing in the queue.

- `void submitTask(TaskDequeue* q, Task* t)`: This method adds a new job to the queue and can be only called by the owner thread.
- `Task* fetchTask(TaskDequeue* q)`: This method removes the next available job from the queue and can be only called by the owner thread. When the `TDQ` is empty it returns `NULL`.
- `Task* fetchTaskFromOthers(TaskDequeue *q)`: This method allows a core to remove the job at the end of another core's queue. This method will not be called by the owner thread. When the `TDQ` is empty it returns `NULL`. Please note that the `fetchTaskFromOthers` and `fetchTask` methods are designed operate on opposite ends of the queue.

  These method declarations are provided in the file `"multi_sched.h"`. You must complete the implementations in a separate file `"multi_sched.c"`.You are not allowed to make changes in the method signatures of these API methods. However, you may add other methods or structs to `"multi_sched.h"` and `"multi_sched.c"` for your needs.

2.3 Core Simulator Threads

Core simulator threads will be executing a single method called `processJobs`. You will implement this method to manage scheduling of jobs. You might find the declaration of this method in `"simulator.c"` with the following signature:

```
void processJobs(void* arg)
```

where the argument *arg* contains the `TDQ` and it's core ID. Inside the file `sdd`, you are given the input struct definition for this method as:

```
typedef struct ThreadArguments {
    TaskDequeue* q;
    int id;
} ThreadArguments;
```

Simulator cores will run a processing loop inside `processJobs`, where tasks will be fetched from its own `TDQ` or other cores and then executed. The method that will simulate the job execution will be provided to you under the name `executeJob`. This method will also update the remaining job duration considering the cache affinity coefficient. We provide a sample `executeJob` implementation in the `"sim_methods.c"` file so that you can test your simulator. In this sample implementation, threads simply sleep for simulating an execution.

Each core simulation instance will run `processJobs` method in a separate independent thread. Inside the main loop, the core will decide to fetch a job from its own `TDQ` or other queues. Other queues might be found using the global variable `global_queues`. When a core is idle or has fewer jobs, it can borrow from other cores to ensure a balanced workload. Here's how the method should work:

- **Monitor Job Load on the Core**: Each core periodically checks the state of its own `TDQ` whether it is heavily loaded or under utilized.
- **Dynamic Load Balancing Using `fetchTaskFromOthers()`**: If the queue contains few or no jobs, the core may initiate load balancing by seeking jobs from other cores' queues. From this point on, ownership of the job must be transferred from the old core to the new core so that only one core can execute this task.

- **Job Processing**: After handling load balancing, it should call `executeJob` method where job execution is simulated. This method signals whether the given job has finished its execution with its return value. You should use this return value. When the execution period finishes, the thread should check if any of the dependents of the terminated job are ready to run (i.e all prerequisite tasks are finished). If that is the case, the thread should take that job into its own queue. A job should be inserted back to `TDQ` belonging to this core if it still has remaining time.
- **Dependent Tracking**: After a job is finished, we already discussed that the other jobs relying on the finished job should be traversed and their prerequisite counter should be updated. If this number has reached 0, only then that job should be in the pool for scheduling. If a job has prerequisites being processed, it should not be even considered for being scheduled on a core. At any given time, the jobs in the queues of the processors should have their prerequisite counter equal to 0.
- **Monitor Termination**: `executeJob` will signal if a job is finished. If all jobs are finished, main thread will set the flag *stop_threads*. Each core should check this flag and terminate `processJobs` if flag is raised. If you don't terminate `processJobs` correctly, program will not terminate or some jobs will not be finished.

This procedure is created for each core as a thread from the main. The initial distribution of the root jobs across cores will be given to you in the main method. When your simulator starts, each core will have a set of preloaded jobs in its `TDQ` . Hence, initially, some queues might be heavily loaded while others run empty. To address this imbalance, cores should periodically check if they need to retrieve tasks from others using the `fetchTaskFromOthers` API and do balancing. You are not allowed to make changes in the main method of `"sim_methods.c"`. You only need to focus on processing jobs and implementing dynamic load balancing as cores become idle or overloaded inside `processJobs`.

A suggested approach for balancing is to set two threshold values for the queue size as explained below:

- **High Watermark (HW)**: If a core's queue size exceeds a certain

task count (e.g., 20 tasks), it may ask help from other cores to fetch tasks from its own queue.

- **Low Watermark (LW)**: If a core's queue size drops below this level (e.g., 10 tasks), it can fetch tasks from cores which requires help by using `fetchTasksFromOthers`.

However, you are free to design your own load balancing algorithm. Keep in mind that when a task is moved to another core, it loses its cache affinity, which may impact efficiency. In your report, you should clearly explain your load balancing design.

2.4 Ensuring Synchronization

Proper synchronization of `TDQ` is crucial since multiple cores may access their own or each other's queues concurrently. This requires you to handle shared data carefully, as concurrent reads and writes could lead to race conditions, where the outcome depends on the order of operations. For example, two cores may try to fetch the same task or enqueue tasks in an inconsistent state. Without effective synchronization, various issues can arise:

- Double insertion and deletion: Two cores could accidentally fetch the same job, where both cores fetches a job from head of another queue by using `fetchTaskFromOthers`. They may fetch the same job and when they finished for the cycle they submit same job to their queues causing duplicate jobs.
- Lost Insertion/Deletion: `TDQ` can be corrupted when its core tries to submit a job to the tail of the queue after a cycle finished with `submitTask` and other queue tries to fetch a task from tail of the queue with `fetchTaskFromOthers`. This can cause lost jobs.

To prevent these issues, you must implement a synchronization mechanism of your choice, such as mutexes, atomic operations, or other concurrency control primitives. Synchronization ensures that critical sections are atomic, maintaining data integrity and preventing errors like duplicated tasks or data corruption.

However, you must balance safety with performance, as too much synchronization can create bottlenecks, reducing overall efficiency. Using

heavy locking (like a single mutex) around the entire `TDQ` method bodies ensure safety but restricts concurrency as it serializes whole queue access. Serialization can severely degrade performance in a multi-core system where concurrent processing is expected. You should remember that mutexes are expensive, and consider approaches that minimize synchronization overhead and the size of synchronized code blocks. You may use lock-free or fine-grained solutions improve performance, but it can increase implementation complexity.

In your report, you should clearly describe your chosen synchronization method, explain how it prevents the above issues, and provide formal arguments supporting the correctness of your approach.

3 Submission Guidelines

In this PA, we expect you to submit three C header and implementation files along with your report.

- **multi_sched.h**: C header file containing the `TDQ` and its API declarations. Make sure to include your changes in `TDQ` template we provided.
- **multi_sched.c**: C file that contains your implementation of `TDQ` . Make sure that this file and it's header compiles without dependency to the main file. We will compile it with different main files for testing purposes.
- **report.pdf**: Your report that explains your `TDQ` implementation. Answer the following questions in the report: 1. How did you ensure thread safety in your `TDQ` implementation? 2. How does your load balancing scheme work? How do threads decide to steal jobs or give away their jobs? 3. What is the internal scheduling policy you used in a core? Why did you choose that? 4. If we were not allowed to use mutexes, how could we make sure the `in_degree` decrements were carried out robustly in the `Task` data structure?

During the submission of this homework, you will see two different sections on SUCourse. You are expected to submit your files separately.

You should NOT zip any of your files. Please submit your report.pdf to "PA2 – Report Submission" and your three C files to "PA2 – Code Submission". The files that you submit should NOT contain your name or your id. SUCourse will not accept if your files are in another format, so please be careful.

IMPORTANT NOTE: If the file names, struct names or the public method signatures do not match with the format explained above, you might get 0 from this assignment since the automated tests will fail.

IMPORTANT NOTE 2: In order to compile and test your implementation, please use the **makefile** provided in the PA2 bundle or use the commands in this files. We will use these commands for compiling your implementation during the automated grading and evaluation.

4 Grading

- **Compilation and termination (10 pts):** Given the `"makefile"` file that uses your `"sdd.c"`, when it is compiled with the command:

  ```
  make sim
  ```

  it successfully generates the executable file `"sim"`. Moreover, all the tests successfully terminate. Successful termination entails that the program never deadlocks.

- **Single thread `TDQ` implementation (10 pts):** Your queue implementation works correctly in the case of sequential access of single thread. Even though there will be a single thread, we will use all three API methods in the tests.

- **Multi-thread `TDQ` implementation (10 pts):** Your queue implementation works correctly in the case of concurrent access of multiple threads. In this case, we will obey the API restrictions on the methods threads might call. We will check for job duplication and lost while evaluating the correctness of your queue.

- **Single core Simulation (10 pts):** Your `processJobs` implementation will be tested when simulation runs with core number set to 1.

- **Multi core Simulation (30 pts):** Your `processJobs` implementation will be tested when simulation runs with multiple cores. Again, we will consider the job duplication or loss as the criteria of correctness.
- **Performance tests (10 pts):** We expect your load balancing scheme to be efficient. Therefore, we will measure virtual runtime in your test runs. That is, we will estimate the time a real multicore system would take to run the given tasks using your algorithm. Then, your runtime will be compared against your peers with correct implementations. You will get decreasing points depending on how much better the fastest implementation was compared to yours.
- **Report (20 pts):** We expect you to submit a detailed report summarizing your work. Your report should contain the following information:
  * How did you ensure thread safety in your `TDQ` implementation?
  * How does your load balancing scheme work? How do threads decide to steal jobs or give away their jobs?
  * What is the internal scheduling policy you used in a core? Why did you choose that?
  * If we were not allowed to use mutexes, how could we make sure the `in_degree` decrements were carried out robustly in the `Task` data structure?

5 Sample Runs

5.1 Sample outputs

In the PA2 bundle, you will find 3 sample task DAGs with *sample_task*.txt* files. Additionally, we have provided the expected simulator outputs for these task files in the *sample*.txt*. Keep in mind that order of executions may change. Therefore, your program's output may not exactly match the order of the given sample output. However, you

must make sure all jobs are finished and there are no lost jobs. Also, same job should not be run twice.

At the end program will exit and print:

All tasks finished, joining threads

This line points graceful termination of program. Absence of this line may point deadlock or abrupt termination. You may refer to first point of grading criteria for graceful termination.

IMPORTANT NOTE: We will use automatic grading to evaluate your outputs. **You should not add any extra output messages** as it may disrupt automatic grading.